



**Université Abderrahmane Mira –
Béjaïa**

**Faculté des Sciences Exactes
Département d'Informatique**



Rapport de Projet

Titre du Rapport

- Systeme de Gestion de Cabinet Medical -

Réalisé par :
Yacine MADANI

Encadré par :
Mme. N.YESSAD
(*Enseignante*)

Présenté dans le cadre du : **Projet du module Génie Logiciel**
Niveau : **Troisième année Licence Informatique (L3)**
Spécialité : **Systèmes Informatiques**
Année Universitaire : **2025–2026**

Systeme de Gestion de Cabinet Medical

Yacine Madani

November 2025

Introduction

Dans le contexte actuel, la gestion efficace d'un cabinet médical constitue un enjeu majeur pour garantir la qualité des soins et la satisfaction des patients. Les tâches administratives telles que la planification des rendez-vous, le suivi des dossiers médicaux ou la gestion des consultations peuvent rapidement devenir complexes et chronophages lorsqu'elles sont effectuées manuellement.

Le projet présenté dans ce rapport consiste à concevoir et développer un système de gestion pour un cabinet médical, capable de centraliser et d'automatiser ces opérations. L'objectif principal est de fournir un outil fiable, sécurisé et facile à utiliser, qui permette au personnel médical de se concentrer sur les soins tout en optimisant l'organisation interne du cabinet.

Ce rapport détaille les différentes étapes du projet, depuis l'analyse des besoins jusqu'au développement du système, en passant par la conception de la base de données et de l'interface utilisateur. Il met également en évidence les bénéfices attendus pour le cabinet médical ainsi que les perspectives d'évolution du système.

Description

ClinicaPro le logiciel conçu pour la gestion d'un cabinet médical. Il offre au médecin un outil puissant pour l'informatisation de son activité, facilitant ainsi la gestion des différentes tâches du quotidien.

Objectifs du logiciel

- Proposer un logiciel généralisé pour tout médecin indépendamment de sa spécialité.
- Une interface facile à manipuler éliminant toute confusion possible.
- Mettre en place les différentes fonctionnalités exigées par le médecin.
- L'automatisation de certaines tâches selon le choix.

Informations techniques

Nom du logiciel : ClinicaPro

Version : 1.0.0

Type d'application : Application Desktop

Langage de programmation : Java

Modèle de conception : Architecture trois tiers

Paradigme utilisé : Programmation orientée objet (POO)

IDE : Visual Studio Code (VS Code)

Suivi du projet : GitHub

Base de données :

- MySQL
- phpMyAdmin pour l'administration et le suivi de la base
- Intégration via JDBC / MySQL Connector

Analyse des besoins / de l'existant

Les informations qui suivent représentent une synthèse d'une recherche menée auprès de différents médecins :

- **Dr. TAMASOUT N : Dentiste**
- **Dr. SAIDANI L : Hépto-Gastro-Entérologue**
- **Dr. S OUKACHBI Epse IBRIR : Labo d'analyse médicales**

dans le but d'explorer le métier concerné et de collecter toutes les informations susceptibles d'améliorer le cahier des charges. Après le lancement d'un questionnaire et l'analyse de différentes interfaces fonctionnelles et professionnelles, voici les détails rassemblés :

Gestion des patients

Informations : IdPatient, nom, prénom, âge, sexe, groupeSanguin, adresse, téléphone.

Comportement :

- Prendre rendez-vous
- Annuler rendez-vous
- Voir dossier
- Régler facture

Traitement :

- Ajout ou suppression d'un patient
- Modifier les informations de tous ou de certains patients
- Vérifier l'existence d'un patient
- Afficher le nombre de patients actuel
- Afficher le dossier du patient

Gestion médecin

Informations : IdMédecin, nom, prénom, âge, sexe, groupeSanguin, adresse, téléphone.

Comportement :

- Soigner
- Diagnostiquer
- Délivrer une ordonnance
- Suivre le règlement des factures

Traitement :

- Modifier les informations du médecin
- Session spéciale médecin
- Vérifier ses coordonnées lors de l'authentification
- Fermer sa session automatiquement après la fin de la journée

Gestion des ordonnances

Informations : numOrdonnance, description, médicament, analyse, radio, dateOrdonnance.

Comportement :

- Ordonnance type
- Historique

Traitement :

- Créer une nouvelle ordonnance
- Supprimer une ordonnance existante

- Rectifier / modifier une ordonnance
- Afficher une ordonnance
- Imprimer une ordonnance

Gestion du paiement / facturation / tarification

Facture :

Informations : IDFacture, montantsÉlémentaires, dateFacture, montantTotal, datePrévue.

Paiement : Représente le règlement d'une facture, avec : montantVersé, dateVersement.

Tarifs : Fixés généralement par le médecin ou par calcul, associés comme attribut.

Traitement :

- Calcul du montant total à payer
- Créer une nouvelle facture
- Suppression ou modification
- Impression
- Historique

Gestion des rendez-vous

Informations : numRdv, nom, prénom, statut, motif, actes, datePassage, datePrise.

Comportement :

- Historique

Traitement :

- Nouveau rendez-vous
- Annuler / modifier
- Afficher le nombre de rendez-vous

Gestion de réceptionniste

Informations : IdEmployé, nom, prénom, âge, sexe, adresse, téléphone, horaires, salaire.

Comportement :

- Accueillir les patients
- Gérer les appels téléphoniques
- Organiser les rendez-vous
- Suivre et transmettre les informations administratives

Traitement :

- Ajout ou suppression d'un employé
- Modifier les informations d'un employé
- Gérer les horaires et la présence
- Vérifier les données lors de l'authentification

Détails Importants

- Un patient ne peut pas voir son dossier médical sans consultation du médecin.
- Un patient peut prendre/annuler un ou des rendez-vous, qui doivent être confirmés par le/la réceptionniste.
- Un patient peut régler (payer) une ou des facture(s) par tranches.
- Un patient peut avoir une ou des ordonnance(s)
- Un médecin effectue une consultation à la fois.
- Un médecin peut effectuer au minimum une consultation par jour (selon les rendez-vous et disponibilité).
- Un médecin prescrit une ou des ordonnance(s).
- Un médecin génère une ou des facture(s).
- Un médecin (et seulement celui-ci) peut suivre le règlement des factures.
- Une ordonnance peut être prescrite par un et un seul médecin.
- Une ordonnance est destinée à un et un seul patient.
- Une ordonnance type peut être générée.
- Une ordonnance peut être imprimée.
- Un historique des ordonnances doit exister.
- Une facture est générée par un seul médecin.
- Une facture est destinée à un seul patient.
- Une facture doit être réglée avant la date prévue pour le paiement.
- Une facture est réglée par son patient concerné, et si jamais quelqu'un d'autre va la régler le paiement sera enregistré sur le nom du patient concerné.
- Un historique des factures doit exister.
- Une facture peut être imprimée.
- Un employé (réceptionniste dans notre cas) est géré par le médecin (ajout, suppression...etc).
- Un employé doit avoir sa propre session sur le logiciel, et de ce fait, les droits d'accès seront gérés à certaines parties.
- Un employé (et même le médecin) doit s'authentifier avant de pouvoir utiliser le logiciel.
- Un réceptionniste n'a accès qu'à la gestion des rendez-vous et celle-ci n'est pas par le médecin.

Fonctionnalités (Diagramme de cas d'utilisation)

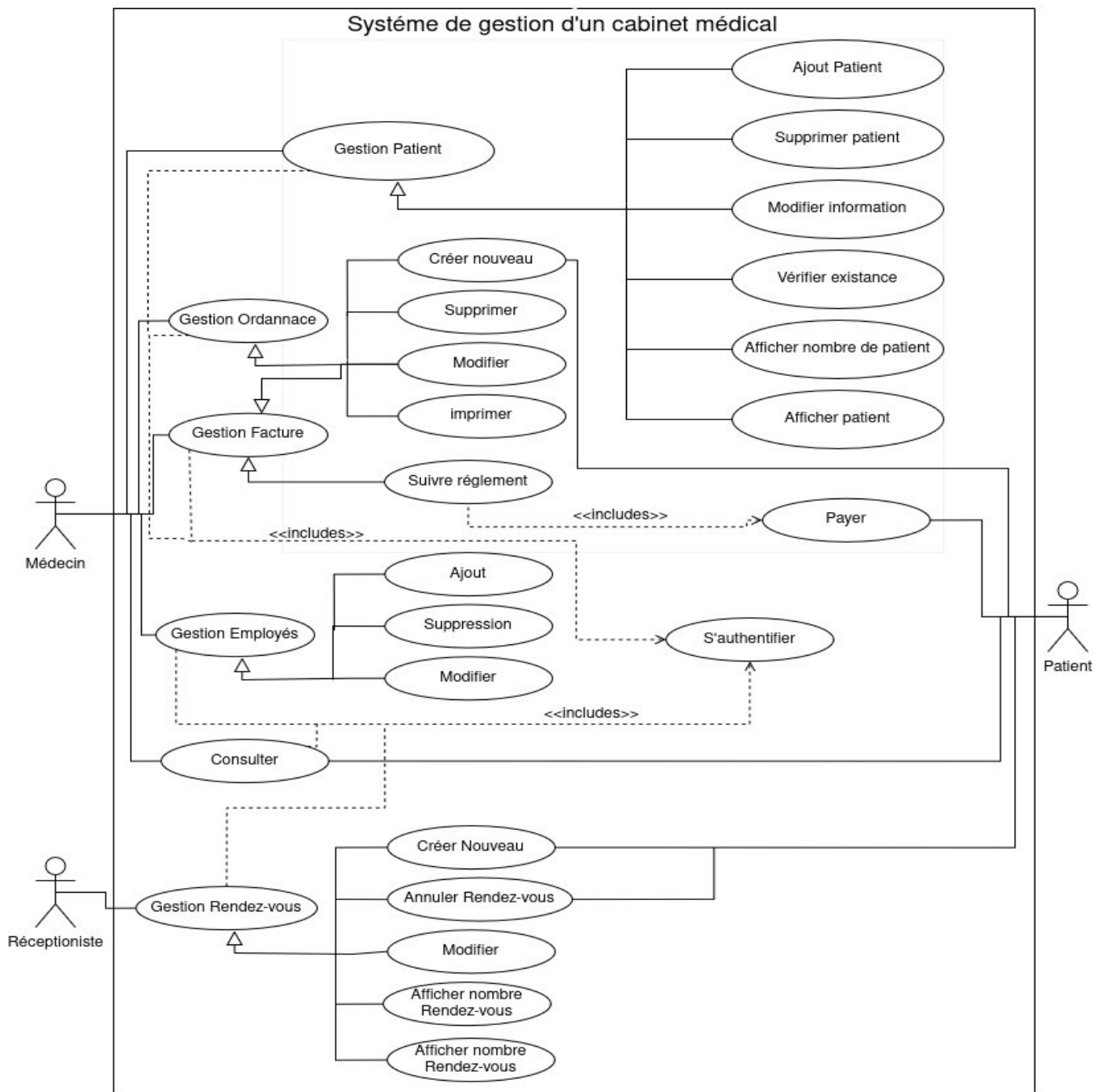
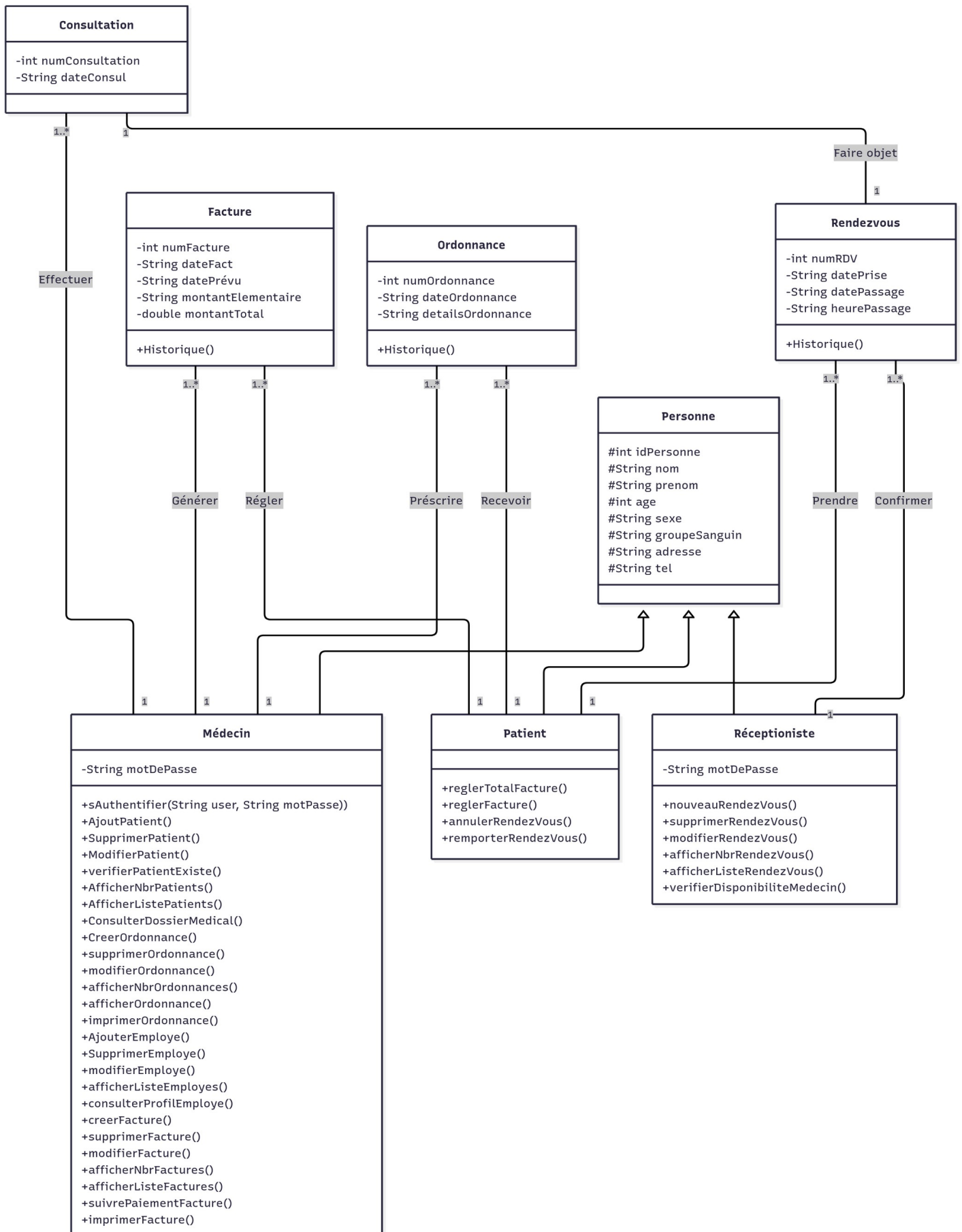
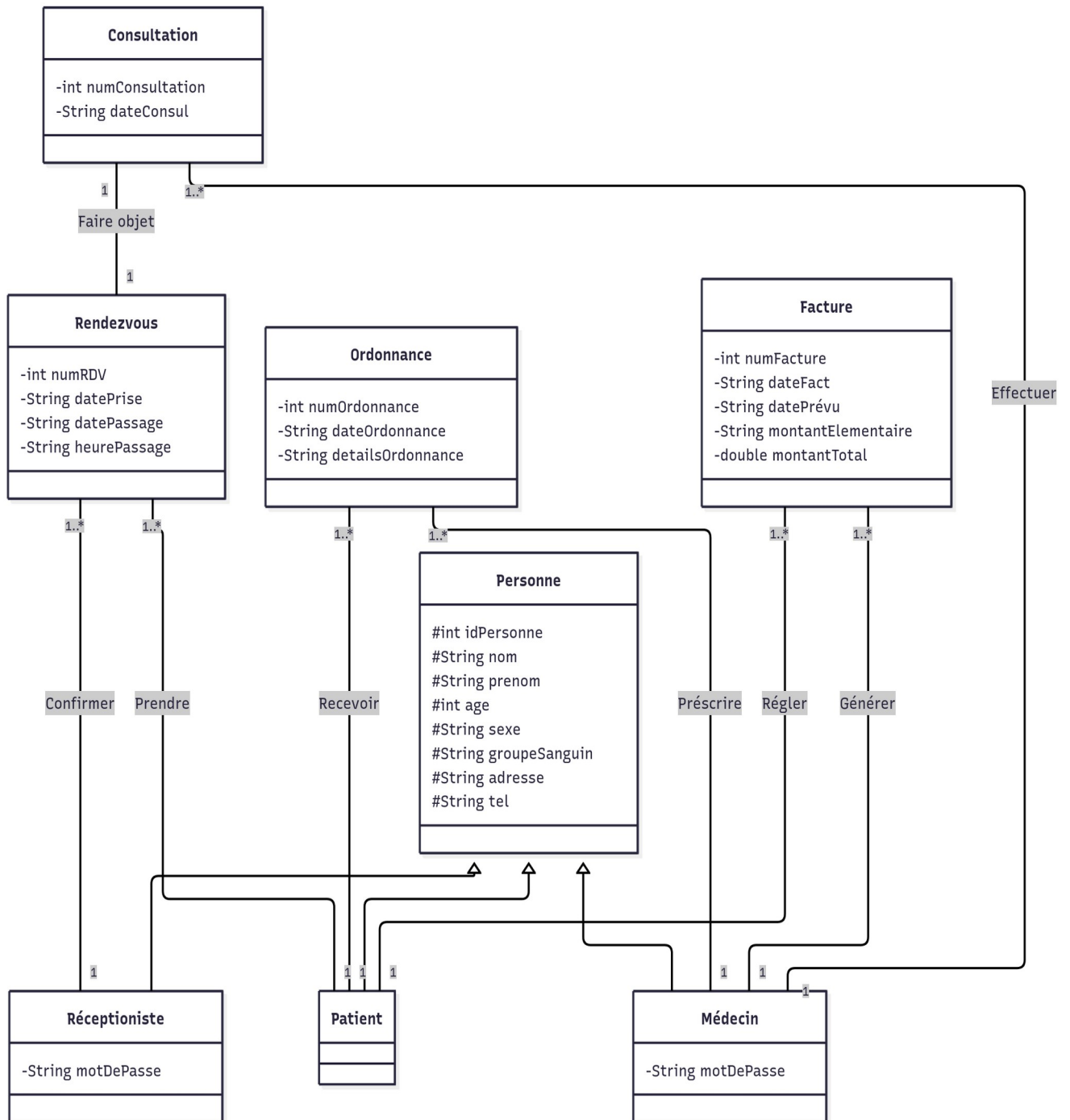


Diagramme de classes



Modèle de domaines



Modèle relationnel

À partir du modèle de domaine que j'avais préalablement défini, j'ai pu construire le modèle relationnel correspondant. Ce dernier traduit les entités et leurs relations en tables normalisées, tout en conservant l'intégrité des données. Chaque relation a été soigneusement pensée pour refléter fidèlement les dépendances et associations identifiées dans le modèle de domaine. Ainsi, le modèle relationnel constitue une base solide pour l'implémentation de la base de données.

- **Personne**(idPersonne, typePersonne, nom, prenom, age, sexe, groupeSanguin, adresse, tel)
- **RendezVous**(numRDV, datePrise, datePassage, #idRéceptioniste, #idPatient)
- **Consultation**(numConsul, dateConsul, #idMédecin, #numRDV)
- **Ordonnance**(numOrdo, détails, datePréscription, #idMédecin, #idPatient)
- **Facture**(numFact, montantElem, montantTotal, dateFact, datePrévu, #idMédecin, #id-Patient, MontantVersé, RestàPayer, dateVersement)

Script SQL

```
1  -- phpMyAdmin SQL Dump
2  -- version 5.2.1
3  -- Host: localhost
4  -- Server version: 10.4.32-MariaDB
5
6  SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
7  START TRANSACTION;
8  SET time_zone = "+00:00";
9
10 /*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
11 /*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
12 /*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
13 /*!40101 SET NAMES utf8mb4 */;
14
15 -- Database: 'ClinicaPro'
16
17 -- Table structure for table 'Consultation'
18 CREATE TABLE 'Consultation' (
19   'numConsul' int(11) NOT NULL,
20   'idMedecin' int(11) NOT NULL,
21   'numRDV' int(11) NOT NULL,
22   'dateConsul' date NOT NULL
23 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
24
25 -- Table structure for table 'Facture'
26 CREATE TABLE 'Facture' (
27   'numFact' int(11) NOT NULL,
28   'idMedecin' int(11) NOT NULL,
29   'idPatient' int(11) NOT NULL,
30   'dateFact' date NOT NULL,
31   'datePrevu' date NOT NULL,
32   'montantElem' text NOT NULL,
33   'montantTotal' int(11) NOT NULL,
34   'montantVerse' int(11) DEFAULT NULL,
35   'dateVerse' date DEFAULT NULL,
36   'restAP' int(11) DEFAULT NULL
37 );
38
39 -- Table structure for table 'Ordonnance'
40 CREATE TABLE 'Ordonnance' (
41   'numOrdonnance' int(11) NOT NULL,
42   'idMedecin' int(11) NOT NULL,
43   'idPatient' int(11) NOT NULL,
44   'details' text DEFAULT NULL,
45   'datePresc' date NOT NULL
46 );
47
48 -- Table structure for table 'Personne'
49 CREATE TABLE 'Personne' (
```

```

30 'idPersonne' int(11) NOT NULL,
31 'typePersonne' varchar(30) NOT NULL,
32 'nom' varchar(40) NOT NULL,
33 'prenom' varchar(40) NOT NULL,
34 'age' int(11) NOT NULL,
35 'sexe' varchar(20) NOT NULL,
36 'groupeSanguin' varchar(3) NOT NULL,
37 'adresse' varchar(50) NOT NULL,
38 'tel' int(11) NOT NULL
39 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
40
41 -- Sample data for 'Personne'
42 INSERT INTO 'Personne' ('idPersonne', 'typePersonne', 'nom', '
    prenom', 'age', 'sexe', 'groupeSanguin', 'adresse', 'tel')
    VALUES
43 (1, 'Medecin', 'Madani', 'Yacine', 20, 'masculin', 'A+', 'Tazmalt',
    1490494);
44
45 -- Table structure for table 'RendezVous'
46 CREATE TABLE 'RendezVous' (
47 'idRdv' int(11) NOT NULL,
48 'idRecep' int(11) NOT NULL,
49 'idPatient' int(11) NOT NULL,
50 'datePrise' date NOT NULL,
51 'datePassage' date NOT NULL
52 );
53
54 -- Indexes and constraints
55 ALTER TABLE 'Consultation'
56 ADD PRIMARY KEY ('numConsul'),
57 ADD KEY 'fk_idmedecin' ('idMedecin'),
58 ADD KEY 'fk_rendezvous' ('numRDV');
59
60 ALTER TABLE 'Facture'
61 ADD PRIMARY KEY ('numFact'),
62 ADD KEY 'idMedecin' ('idMedecin'),
63 ADD KEY 'idPatient' ('idPatient');
64
65 ALTER TABLE 'Ordonnance'
66 ADD PRIMARY KEY ('numOrdonnance'),
67 ADD KEY 'idMedecin' ('idMedecin'),
68 ADD KEY 'idPatient' ('idPatient');
69
70 ALTER TABLE 'Personne'
71 ADD PRIMARY KEY ('idPersonne');
72
73 ALTER TABLE 'RendezVous'
74 ADD PRIMARY KEY ('idRdv'),
75 ADD KEY 'fk_recep' ('idRecep'),
76 ADD KEY 'fk_patient' ('idPatient');
77

```

```

98 ALTER TABLE 'Personne'
99     MODIFY 'idPersonne' int(11) NOT NULL AUTO_INCREMENT ,
        AUTO_INCREMENT=2;
100
101 ALTER TABLE 'Consultation'
102     ADD CONSTRAINT 'fk_idmedecin' FOREIGN KEY ('idMedecin')
        REFERENCES 'Personne' ('idPersonne'),
103     ADD CONSTRAINT 'fk_rendezvous' FOREIGN KEY ('numRDV') REFERENCES
        'RendezVous' ('idRdv') ON DELETE CASCADE ON UPDATE CASCADE;
104
105 ALTER TABLE 'Facture'
106     ADD CONSTRAINT 'Facture_ibfk_1' FOREIGN KEY ('idMedecin')
        REFERENCES 'Personne' ('idPersonne'),
107     ADD CONSTRAINT 'Facture_ibfk_2' FOREIGN KEY ('idPatient')
        REFERENCES 'Personne' ('idPersonne') ON DELETE CASCADE ON
        UPDATE CASCADE;
108
109 ALTER TABLE 'Ordonnance'
110     ADD CONSTRAINT 'Ordonnance_ibfk_1' FOREIGN KEY ('idMedecin')
        REFERENCES 'Personne' ('idPersonne'),
111     ADD CONSTRAINT 'Ordonnance_ibfk_2' FOREIGN KEY ('idPatient')
        REFERENCES 'Personne' ('idPersonne') ON DELETE CASCADE ON
        UPDATE CASCADE;
112
113 ALTER TABLE 'RendezVous'
114     ADD CONSTRAINT 'fk_patient' FOREIGN KEY ('idPatient') REFERENCES
        'Personne' ('idPersonne'),
115     ADD CONSTRAINT 'fk_recep' FOREIGN KEY ('idRecep') REFERENCES '
        Personne' ('idPersonne');
116
117 COMMIT;
118
119 /*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
120 /*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
121 /*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;

```

Listing 1 – Dump SQL complet de la base ClinicaPro

Conclusion

La réalisation de ce système de gestion pour un cabinet médical a permis d'apporter une solution fiable, structurée et adaptée aux besoins réels du personnel soignant. Grâce à l'automatisation des tâches essentielles — telles que la gestion des dossiers patients, la planification des rendez-vous, le suivi des consultations et la génération des documents médicaux — le système contribue à améliorer l'efficacité du travail quotidien tout en réduisant le risque d'erreurs.

Ce projet m'a également permis de mettre en pratique des concepts fondamentaux en développement logiciel, notamment l'analyse des besoins, la modélisation, la conception d'interfaces et la gestion de bases de données. Il a constitué une occasion d'acquérir une vision plus concrète de l'importance de l'ergonomie, de la sécurité des données sensibles et de la fiabilité dans les systèmes destinés au domaine médical.

En définitive, le système développé représente une base solide qui pourra être améliorée et enrichie par de futures fonctionnalités, telles que l'intégration de notifications, la gestion avancée des statistiques ou encore une interface web. Ce projet démontre l'intérêt et la nécessité d'outils numériques efficaces pour optimiser la qualité du service au sein des structures médicales.